

The QEC Challenge

A public, automatically verified leaderboard for quantum low-density parity-check codes

Unitary Foundation

github.com/unitaryfoundation/qldpc-challenge

October 1, 2026

Abstract

The QEC Challenge is a public leaderboard for quantum low-density parity-check codes. A participant submits a single JSON file describing one code — a CSS code, or since schema 0.4 a general stabilizer code — and continuous integration recomputes the code’s properties, actively tries to refute its distance claim, and merges onto the board only a code that holds up. Three principles shape the design. (i) *Everything cheap is trustless and mandatory*: the verifier recomputes n , k , commutation, connectivity, check weight, and the geometric locality class from scratch over $\mathbb{F}_2$ — a wrong claimed k or a non-commuting “stabilizer” is rejected in milliseconds. (ii) *Distance is layered by how much is proven*: a submitter attaches an explicit logical-operator *witness* that proves $d \leq$ value with no trust required; an adversarial *refutation gate* then searches for a lighter logical, and separate certification tiers upgrade a claim to *corroborated* or *exact*. The same discipline extends to the circuit level: an entry may ship its syndrome-extraction circuits and claim a witnessed circuit-level distance and a measured logical error rate, both re-derived and re-measured by CI. (iii) *A code is not a single number*: the primary tracks are a grid of *computed* axes (locality class $\times$ check-weight class, with CSS and general stabilizer codes on separate boards), and within each cell the ranking is a Pareto frontier over (n, k, d, w) rather than a scalar; two efficiency figures — the *operational* efficiency kd^2/n and, for codes shipping a verified layout, the *geometric* efficiency $g = 4kd^2/(np^2r^4)$, normalized so the planar surface code scores exactly 1 — are reported alongside the frontier, never in place of it. The whole pipeline — validator, refutation gate, certifiers, circuit-tier verifier, scorer, static-site generator — lives in the repository and runs in CI on every pull request, which also makes the board usable as the verification harness of automated (LLM- and agent-driven) code-discovery loops. As of 1 October 2026 the board holds 1791 verified codes, 625 of them certified exact.

1 Overview

A submission is one JSON file placed under `codes/`, conforming to `schema/code.schema.json` (described in `schema/SCHEMA.md`). A contributor opens a pull request adding the file (one new code per PR, enforced by CI; an entry’s circuit artifacts count as part of the same submission); GitHub Actions runs the verifier (`verify/qldpc_verify.py`) and the distance refutation gate (`verify/gate_changed.py`), and a green check means the code’s cheap properties are confirmed *and* an adversarial search failed to find a logical lighter than the claimed distance. After merge, `site/build.py` regenerates the static leaderboard into `docs/` automatically (an index with per-cell Pareto tables and plots, a separate board for general stabilizer codes, a logical-error-rate frontier, a page per code, a reference list, and status badges). The board is seeded with literature baselines — the planar open-boundary codes of Liang, Eberhardt, and Chen [1], the bivariate-bicycle gross code [2], generalized-bicycle codes [3, 4], coset codes [5], the non-abelian lifted-product mitten codes [6], and the surface, toric, and Steane codes [7, 8] — so a new entry is always measured against published codes rather than an empty board.

Two things are deliberately *not* asked of the submitter: a proof that their distance is exact (that is the server’s job), and a single headline rank (the board is a Pareto frontier). Everything else — the parity checks, the claimed n, k, d with witnesses, an optional layout, an optional circuit tier, and provenance — is supplied and then recomputed, witnessed, or attacked by the verifier.

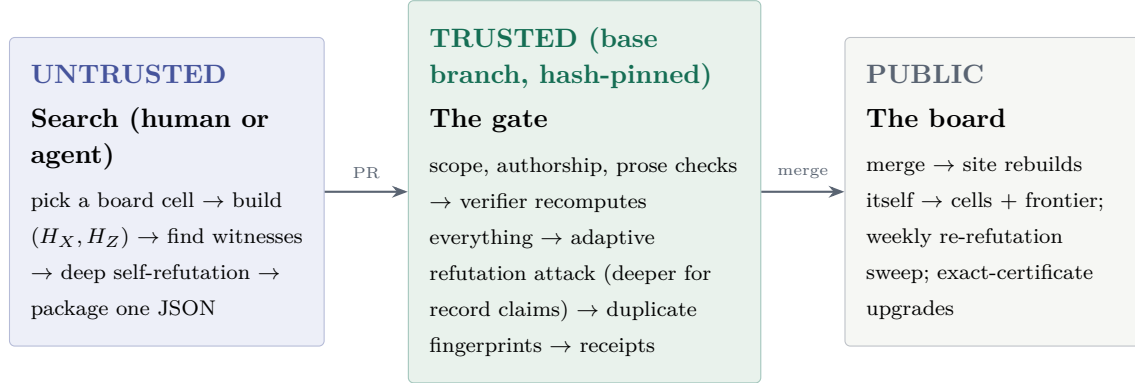


Figure 1: Three zones, one direction of trust. A submission originates in an untrusted search (increasingly, an automated one), is judged entirely by pinned code running from the base branch, and only then becomes part of the public record — which keeps being re-attacked after merge.

2 Why a leaderboard of cells, not one number

A quantum code trades several quantities against one another — physical qubits n , logical qubits k , distance d , maximum check weight w , and geometric locality — and separately has a decoding performance. Collapsing all of that into one scalar would reward gaming one axis at the expense of the rest. So the board is stratified in three layers (TRACKS.md).

Layer 1: primary tracks, computed, never self-declared. The primary tracks are a grid of two nested axes whose membership is *derived by the verifier* from the parity checks and the layout, so a track cannot be claimed by relabeling:

- a **locality class**: `local-2d-single` (one layer, interaction radius ≤ 4) $\subset$ `local-2d-bilayer` (up to two layers — the flip-chip regime — radius ≤ 7) $\subset$ `unrestricted` (everything else, including any code without a layout and any three-dimensional layout);
- a **check-weight class**: `weight-4` $\subset$ `weight-6` $\subset$ `weight-8` $\subset$ any weight, from the maximum row weight of H_X, H_Z (for a general stabilizer code, the number of qubits a generator acts on, a Y counting once).

The classes nest: a single-layer weight-4 code also competes on every looser board. Each (locality, weight) cell is a board of its own (Figure 2). The code type is a third cell dimension: CSS codes and general stabilizer codes are ranked on separate leaderboards with the same grid, because kd^2/n does not mean the same thing for both — the two families decode differently, and how the figure tracks a logical error rate is less settled for non-CSS codes. A stabilizer code never dominates or is dominated by a CSS entry.

Layer 2: family tags, filterable, never ranked. The construction family (bivariate bicycle, generalized bicycle — including non-abelian two-block group-algebra constructions [9] — coset,

		check weight			
		$w \leq 4$	$w \leq 6$	$w \leq 8$	any w
locality	single	surface, packings		planar $[[288, 8, 15]]$	
	bilayer		BB $[[360, 12, 24]]$		LP $[[150, 30, 10]]$
	unrestricted		BB records	coset, 2BGA	GB $w \geq 10$

Figure 2: The primary-track grid: locality class $\times$ check-weight class, both computed from the submission (example occupants of the CSS board shown). Within each cell the ranking is the Pareto frontier over (n, k, d, w) ; a code in a tight cell also competes in every looser cell it nests into. The general-stabilizer board has the same grid.

lifted product, tile, topological, ...) cannot be recovered from the matrices, so it is a self-reported, filterable tag that confers no ranking advantage.

Layer 3: verified flags. Only properties a verifier or a certificate can prove: CSS commutation (or, for a general stabilizer code, that its generators commute), the locality class (which doubles as Layer-1 membership), **modular** (the layout assigns every qubit to a hardware module), and the distance-confidence tier of Section 6.

Diagnostics, displayed but never ranked. The verifier also reports properties that no track reads: the girth of each side’s Tanner graph, the row and column weight profiles, a census of small trapping sets, the Euclidean support diameter of each stored distance witness for a laid-out code, and a heuristic routing cost (nearest-neighbour SWAPs per syndrome-extraction round to connect every check’s support on the shipped layout — a minimum-spanning-tree lower bound, not a schedule). They appear on code pages and as sortable columns; they enter neither the frontier nor any score.

Pareto frontier, not a single winner. Within a cell, a code is a record if no other code in the cell dominates it — no other has $n' \leq n$, $k' \geq k$, $d' \geq d$, $w' \leq w$ with at least one strict inequality. There can be many co-leaders. A record whose *only* strict axis is d is a different kind of claim: both numbers are witness-backed upper bounds, so the comparison says which search ran deeper as much as which code is better, and this is where the board has historically been wrong ($[[882, 18, 30]] \rightarrow 29$, $[[684, 12, 81]] \rightarrow 66$). The gate reports that pattern explicitly, naming the entry that was beaten, so that it can be re-measured at matched depth.

Efficiency as sortable columns, not the rank. Two efficiency figures are reported per cell and sortable, but neither collapses the frontier into one number: the *operational* efficiency kd^2/n used by the 2D-locality literature, and the *geometric* efficiency g of Section 3.1, which additionally prices the layout a code ships with. Both are monotone in d , so a code whose distance is only an upper bound carries the one $d \leq$ marker in the distance column that governs both.

3 Background: two efficiency metrics and the BPT bound

For an $[[n, k, d]]$ stabilizer code, the Bravyi–Poulin–Terhal (BPT) bound [10] states that any geometrically 2D-local code obeys

$$\frac{kd^2}{n} \leq c(r, w, q), \quad (1)$$

where the constant c depends only on the interaction range r , the maximum stabilizer weight w , and the on-site dimension q — not on n . The optimal constant is open, which makes “within a fixed locality model, maximize kd^2/n ” a well-posed target for the 2D-local cells. For asymptotically good codes [11, 12], where k and d both grow linearly with n , kd^2/n grows like n^2 without bound — which is exactly why the board buckets and compares like with like rather than ranking a $[[120, \dots]]$ entry against a $[[10^4, \dots]]$ one. Reference points from the literature, spanning the efficiency range (all $q = 2$; $w = \text{max check weight}$):

Code	$[[n, k, d]]$	topology	w	kd^2/n
Rotated planar surface [7]	$[[d^2, 1, d]]$	planar	4	1
Gross code (IBM) [2]	$[[144, 12, 12]]$	periodic	6	12
Tile code [13]	$[[512, 18, 19]]$	planar	8	12.7
Generalized bicycle [3]	$[[126, 28, 8]]$	periodic	10	14.2
Coset two-block [5]	$[[168, 20, 14]]$	unrestricted	8	23.3
Mitten (lifted product) [6]	$[[540, 108, 18]]$	unrestricted	9	64.8

The planar rows are the open-boundary program of Liang, Eberhardt, and Chen [1] and the tile codes [13]; the coset row is the two-block generalization of Aydin, Tamo, and Barg [5]; the mitten and ZSZ-LP families [6, 14] reach rate $1/5$ at weight 9 by two different non-abelian routes. Whether planar codes can close the gap to the periodic and unrestricted records is the kind of question a live leaderboard can resolve.

3.1 Geometric efficiency: pricing the layout

Operational efficiency deliberately ignores *how* a code is laid out: a periodic code and a planar code with the same $[[n, k, d]]$ score the same kd^2/n , although one assumes long-range couplers the other does not need. The board therefore also reports, for every code whose layout the verifier has accepted, a *geometric* efficiency that prices the layout itself:

$$g = \frac{4kd^2}{n\rho^2r^4}, \quad (2)$$

where r is the *measured* interaction radius of the shipped layout (the maximum Euclidean check diameter, in units of the unit qubit spacing) and ρ its layer count. This is the BPT ratio (1) charged for its geometric resources: the r^4 factor is the range dependence of the BPT constant in 2D, and ρ^2 is the unique exponent that makes re-packaging a layout into layers score-neutral, so g cannot be inflated by stacking. The constant $4 = (\sqrt{2})^4$ normalizes the rotated planar surface code ($r = \sqrt{2}$, $\rho = 1$) to exactly $g = 1$, which gives the figure an absolute meaning: a code family that sustains $g > 1$ at growing distance packs more protected logical content per qubit under the *same* hardware locality budget as the surface code. A layout may also carry three-dimensional coordinates; the BPT bound in D dimensions is $kd^{2/(D-1)} = O(n)$, and the same coarse-graining gives $g = 2\sqrt{2}kd/(n\rho r^3)$ for $D = 3$, normalized so a code saturating $kd = n$ at the nearest-neighbour cubic radius $r = \sqrt{2}$ scores 1. A 3D layout earns no 2D-local class and g is compared within a dimension, never across.

Three design details keep g honest. It is computed only from a verifier-accepted layout (Section 5, V9) — r and ρ are recomputed from the shipped coordinates, never taken from the submission — and a code without a layout simply has no g , which is a certification status rather than a judgment. Constant-distance tilings exceed $g = 1$ trivially (a $[[4, 2, 2]]$ block on one plaquette scores $g = 2$), so the board’s headline figure requires $d \geq 3$. And g inherits the distance tier: computed from an upper-bound d it is an upper bound.

When the first version of this document was written, the rotated surface codes held the ceiling at $g = 1$ and the best contributed layout reached $g \approx 0.38$. That is no longer the state of the board. Multi-band dense packings of surface-code patches on a shared plane, generalizing the code-deformation procedures of Fujiu, Nagayama, Nishio, Kawaguchi, and Satoh (arXiv:2511.06758), keep the surface code’s own weight-4 checks and $r = \sqrt{2}$ radius while sharing boundary ancillas between patches, and they now exceed $g = 1$ at every distance the board has them at: $g = 1.68$ at $d = 3$ ($[[925, 173, 3]]$), 1.35 at $d = 5$ ($[[998, 54, 5]]$), 1.31 at $d = 7$, 1.29 at $d = 9$ (the $[[691, 11, 9]]$ member is certified exact), and 1.27 at $d = 13$ ($[[931, 7, 13]]$). The sequence decreases slowly with d , which turns the open question of Section 10 from “can any layout beat 1” into “where does this family’s g converge, and does the advantage survive at the circuit level, where the shared ancillas also share hook errors”.

4 Submission format

A submission is one JSON object (`schema_version` "0.1" through "0.4"; `code_type` "CSS" or "stabilizer") with:

- **n** — physical qubit count; must match the indices used in **checks**. Admissible blocklengths are $n \leq 700$, or $n \leq 1000$ when the maximum check weight is ≤ 8 and the claimed $d \leq 40$ — a verification-budget rule, since the refutation gate must be able to search meaningfully behind every claim in CI.
- **k** — *claimed* logical qubit count; the verifier recomputes it and requires an exact match.
- **checks.X**, **checks.Z** — the parity checks as sparse supports: each is a list of checks, each check a sorted list of distinct 0-based qubit indices. A general stabilizer code instead ships **checks.S**, a list of generators each given as an X support and a Z support (a qubit in both carries Y); this is the binary symplectic matrix $S = (A | B)$.
- **distance.d** — claimed distance = $\min(d_X, d_Z)$, and per side **distance.X/distance.Z**, each a `{value, confidence, witness}` triple, where **confidence** is "upper_bound" or "exact" and **witness** is the support of a logical operator of that Pauli type and weight **value**. A stabilizer code has no sides: it carries a single **distance.P** whose witness is one Pauli operator and whose **value** is its Pauli weight.
- **locality** (optional) — **coordinates** (one $[x, y]$ or $[x, y, z]$ per qubit), **layers** (required with a layout), and optionally **modules** (one hardware-module id per qubit); the verifier recomputes the interaction radius and derives the locality class from it. A missing layout is not an error — the code simply competes as **unrestricted** — but a dishonest one (crammed sites, or sites closer than the unit spacing) fails verification, and an honest layout above every class cap is demoted to **unrestricted** through a named check rather than silently.
- **circuit** (optional; RFC 0001) — the circuit tier. The entry additionally ships memory experiments under **circuits/<slug>/** (**memory_x.stim**, **memory_z.stim**, and their derived detector error models **.dem**) and claims a per-basis circuit-level distance **d_circ** with a witness, a round count $\geq d$, and the pinned **stim** version. It may also carry **gates**, claimed transversal Clifford gates, and **ler**, a measured logical error rate (Section 7).

- **provenance** — **authors** (the humans, by GitHub handle; the PR author must be listed), **construction** (a reproducible recipe), optional **references**, **date**, **notes**, an optional literature-**novelty** status (a submitter claim), an optional **search_budget** (what the search cost), and an optional **model** field naming the AI system that produced the code, to a specific version — machine-discovered entries are attributed as such.
- **witness_provenance** (optional, schema 0.2) — who found the stored witnesses and how, including a *survival stamp*: the number of independent refutation samples the claim has survived. Refutation credit lives on the witness, not on the code’s author list, so a tightening correction by a third party never rewrites authorship.

(A legacy self-declared **tracks** field is deprecated and ignored for ranking; track membership is computed.) Verify locally before opening the PR (the project uses `uv`):

```
uv run python verify/qldpc_verify.py codes/your-code.json
```

The verifier prints a JSON report — per-check pass/fail, the computed n , k , ranks, weight class, locality class, diagnostics — and an **earned_distance** block giving the tier each side actually earned, and exits 0 iff every required check passes. The `cli/` launcher automates the whole path: `./qldpc submit` turns parity-check or symplectic matrices (however constructed — e.g. with the qLDP library [15]) into a verified, PR-ready file, generating and verifying the circuit tier by default; `./qldpc targets` lists which track cells are open and what a code of a given size needs to enter them; `./qldpc recent` summarizes what landed lately; and `./qldpc reproduce` replays an entry from its reproduction manifest.

5 What the verifier checks

A submission is rejected unless all of the following hold. Checks (V1)–(V10) are polynomial-time and run in CI on every PR; the refutation gate and the certification tiers (Section 6) are layered on top.

- (V1) **Schema and indices.** The document conforms to `code.schema.json`, every qubit index in **checks** is in $[0, n)$, the blocklength is admissible, and the file is within size limits.
- (V2) **No collapsed checks.** No qubit index is repeated within a single check (a repeat would XOR away and silently change the code).
- (V3) **Commutation.** $H_X H_Z^\top = 0$ over $\mathbb{F}_2$ for a CSS code; isotropy $AB^\top + BA^\top = 0$ for a general stabilizer code. A submission typed **stabilizer** whose every generator is pure X or pure Z is a CSS code and is rejected until typed as one.
- (V4) **Connectivity.** The Tanner graph is connected, and — because a redundant row spanning two blocks can join the graph without changing the code — so is the Tanner graph of the reduced row echelon form of the checks: the stabilizer group must not be a direct sum over disjoint qubit sets. A direct sum of two board codes is not a new code; the echelon-form check closed the redundant-row evasion that was tried against the first version of the test.
- (V5) **Logical dimension.** The verifier recomputes $k = n - \text{rank}_{\mathbb{F}_2}(H_X) - \text{rank}_{\mathbb{F}_2}(H_Z)$ (or $k = n - \text{rank } S$), requires it to equal the claimed k , and requires $k \geq 1$.
- (V6) **LDPC weight cap and vocabulary.** The maximum check weight is bounded (the board is for LDPC codes), the family tag is from the controlled vocabulary, and a **model** field names a specific version.
- (V7) **Distance witnesses.** For each provided side, the witness v must (a) have weight equal to the claimed **value**, (b) lie in the kernel of the opposite checks (an X -witness commutes with H_Z ; a Pauli witness commutes with every generator), and (c) be nontrivial, i.e. not in the

rowspan of its own checks (not a stabilizer product). This certifies $d_{\text{side}} \leq \text{value}$ as an *upper bound* with no trust required.

- (V8) **Distance consistency.** The claimed d equals the minimum over the witnessed sides.
- (V9) **Layout honesty and locality class (if a layout is present).** `coordinates` must cover all n qubits; at most `layers` qubits may share a site and distinct sites must be ≥ 1 apart (no cramming — a small radius cannot be faked by collapsing qubits onto a point), and either violation fails the submission; the recomputed maximum check diameter then decides the class. The class is *earned* from the layout, never declared.
- (V10) **Quick refutation.** A short random-information-set search must fail to find a logical lighter than the claim (the full gate of Section 6 runs far deeper).

Every **exact** confidence claim is *downgraded to upper_bound here* and flagged for server certification — the cheap PR check never upgrades a distance to exact. The verifier also files, with every CSS code, the fingerprints of its images under local Hadamards, so that a Hadamard-relabelled copy collides with it whichever code type it is submitted under.

6 Distance: three tiers and an adversarial gate

The distance is the minimum weight over nontrivial logicals; for a CSS code $d = \min(d_X, d_Z)$, for a general stabilizer code the minimum Pauli weight. The problem is NP-hard [16], and the hardness is asymmetric: exhibiting a low-weight logical gives only an *upper* bound, while certifying $d \geq D$ is the hard direction — and a submitter has every incentive to overclaim. Heuristic distance estimates routinely deflate under deeper search, so the challenge layers three confidence tiers (Figure 3) behind an active gate.

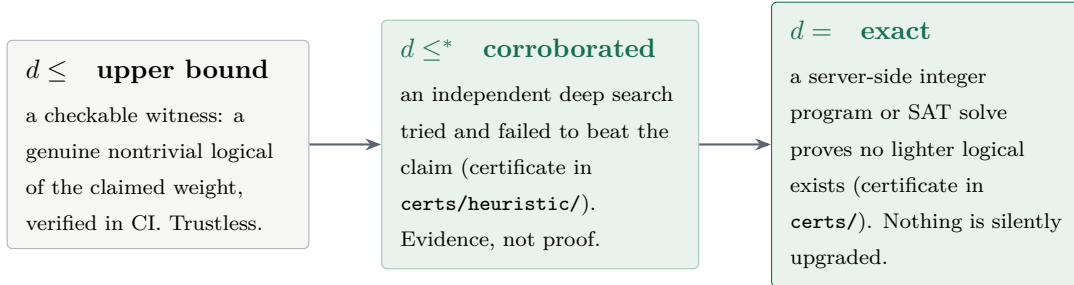


Figure 3: The distance-confidence ladder. Every tier is backed by an artifact in the repository: a witness inside the submission, a heuristic certificate, or an exact certificate.

Upper-bound tier (trustless, in CI). The submitter’s witness is checked exactly as in (V7). A valid witness proves $d_{\text{side}} \leq \text{value}$; the board labels such a code $d \leq$ and lets anyone climb instantly, since the arithmetic is checked rather than trusted.

The refutation gate (adversarial, in CI). Every changed code faces bounded searches for a logical *lighter* than the claim. The gate is staged. A cheap *structural* pass runs first: a circulant generalized-bicycle code, detected from its matrices rather than from its family tag, is attacked through its own block structure, which finds the low-weight logicals brute force misses. Then a random-information-set (RIS) search (randomized distance estimation in the spirit of QDistRnd [17]) runs with an adaptive budget: trials scale with n , and a code that would enter a cell’s Pareto frontier faces a much deeper battery than a dominated one, so over-claims pay extra

scrutiny exactly where gaming would matter. Frontier claims additionally face an accelerated RIS pass (a bit-packed $\mathbb{F}_2$ engine at roughly $150\times$ the baseline trial budget, capped at 90 minutes of wall-clock) that only *proposes* candidates: a find counts as a refutation solely after the pinned Python verifier validates the witness, so the extra depth adds no trust surface, and a missing or broken accelerator falls back to the full multi-seed Python battery rather than a shallower gate. A syndrome-decoder (BP+OSD) search ran alongside RIS for the board’s first year and was retired in September 2026 after a measured comparison showed it never found a logical the RIS search had not, at a small fraction of the trial rate. Any hit is a sound refutation — it exhibits a checkable counterexample — and fails the PR. The per-run seed is random by design, so an over-claim cannot be tuned to evade one fixed search; a weekly whole-board sweep re-runs the refutation with fresh seeds to accumulate coverage over time.

The gate prices the *diff*, not just the code. A layout-only change (checks and distance byte-identical to the base) skips refutation, since the claim already survived when it merged; a tightening correction (values strictly down, new witnesses of exactly the new weights) gets the standard shallow pass; a diff that adds or raises a survival stamp gets the deep battery regardless of record status, because a stamp is itself a claim that deters future refuters; anything else fails closed to the full search. The classification is recomputed from the git diff, never taken from the PR’s framing, and every run leaves a verification receipt as a CI artifact.

Corroborated tier. A maintainer-run deep search (`verify/heuristic_certify.py`) that tries and fails to beat a claim writes a heuristic certificate; the board then shows $d \leq^*$. This is evidence, not proof — the label records exactly how hard the claim has been attacked.

Exact tier (server-certified). To upgrade a side to $d =$, a maintainer proves no lighter non-trivial logical exists. Two certifiers ask the same question. `verify/certify.py` solves one bounded integer program per logical generator t_L of the relevant type (scipy/HiGHS, no commercial solver): does a v exist with

$$Hv \equiv 0, \quad \langle v, t_L \rangle \equiv 1 \pmod{2}, \quad \text{wt}(v) \leq \text{value} - 1, \quad (3)$$

the mod-2 constraints linearized by integer slacks. Its cost is k solves per side, which is why it stalls on high-rate codes. `verify/sat_certify.py` encodes the same question for a SAT solver with one selector variable per generator and a single clause requiring at least one anticommutation, so the whole side is one UNSAT proof instead of k ; because that constraint set is invariant under the block rotations of two-block circulant codes, lex-leader symmetry breaking becomes legal and pays for itself at large k (the $[[126, 28, 8]]$ generalized-bicycle code certifies in minutes where the per-generator form needed hours). If every generator on both sides is proven *infeasible*, d is exact; if a lighter logical is found, the claim is *refuted*; if the solver hits its time limit, the side honestly remains an upper bound. A successful run writes a certificate to `certs/<slug>.json`, which is what upgrades a code’s board label to $d =$. Exact records outrank equal upper-bound ones; nothing is ever silently upgraded. General stabilizer codes are not yet certified — both certifiers minimize Hamming weight per side, not Pauli weight — so every entry on that board is $d \leq$.

7 The circuit tier: distance and error rate under a noise model

A code’s distance is a property of its stabilizer group; what a device experiences is the distance of a particular syndrome-extraction *circuit*, which hook errors and idle noise can lower well below d . The circuit tier (RFC 0001) transplants the code-tier discipline to that question. An entry

ships memory experiments in `stim`'s circuit format for both bases, written in a canonical noise recipe at the reference rate $p = 10^{-3}$ — two-qubit depolarizing noise after every two-qubit gate, reset and measurement flips, and single-qubit depolarizing noise on every idle data qubit in every layer — in which noise placement is not a submitter degree of freedom but the schedule is. Every TICK layer must be genuinely parallel (no qubit operated on twice), since the layer count sets the idle noise and deleting annotations would otherwise shed fault mechanisms. The verifier (`verify/circuit_verify.py`) checks that the noiseless skeleton is deterministic, that the circuit equals its own re-noised skeleton, that the observables are the code's declared logicals on a transversal readout and the detectors match the declared checks, and that the committed detector error model re-derives from the committed circuit under the pinned `stim` mechanism-for-mechanism. The hyperedges of a detector error model are the columns of a parity-check matrix H_{dem} , so the claimed circuit-level distance `d_circ` carries a witness — a set of error mechanisms whose detector sets cancel and whose observable sets do not — checked by the same $\mathbb{F}_2$ arithmetic as (V7), and the CI gate runs the same RIS search on H_{dem} for a lighter undetected logical fault set. The tier is penalty-only: `d_circ` is clamped to $\leq d$, so a circuit can only lower an entry, never raise it. Claimed transversal gates (`circuit.gates`: qubit permutations, a Hadamard or phase gate on every qubit, or a block-wise CNOT) are verified as linear maps on symplectic vectors — the image of every stabilizer must lie in the group and the image of every logical must equal the claimed product modulo stabilizers.

Measured logical error rate. An entry with circuits may also claim a per-round logical error rate. The claim carries `failures`, `shots`, and `rounds`, must meet a failure-count floor, and is decoded with the one pinned decoder (BP+OSD [18], combination-sweep order 10). CI re-derives the detector error model, re-samples an independent replica *sized to discriminate* — enough expected failures that a factor-1.4 under-report is detected at four standard deviations — and fails the claim if the replica disagrees, or if the wall-clock budget truncates the replica so far that it could no longer tell. The site shows a separate frontier over (n, k, LER) that is independent of the (n, k, d, w) frontier: distance and check weight do not enter it. The decoder's bit-exact output is not a promise across platforms, so the tier is statistical by design.

8 Trust architecture and automated discovery

Automated code discovery already works: LLM-guided evolutionary search [19], reinforcement learning [20], and LLM-driven concept evolution over non-abelian families [21] all produce candidate codes far faster than humans can referee them — and an overstated distance is precisely their characteristic failure mode. Most of the board's entries, including the current operational-efficiency record, were machine-discovered and are attributed to the producing model in their provenance. The pipeline is therefore built so that *nothing a submission can contain is trusted*, which is what makes it usable as the verification harness of an autoresearch loop (Figure 1):

- **Scope separation.** A PR that adds code data may not touch the verifier, schema, workflows, or site builder (`check_submission_scope.py`); and for code PRs the entire judgment runs from the *base-branch* checkout, so a submission cannot alter the code it is judged by.
- **Pinned validation stack.** The trusted closure — the verifier, the refutation engine, the GF(2) core, the circuit and error-rate verifiers, the local `validate_candidate` gate, and the integrity checker itself — is pinned by a SHA-256 manifest; any drift fails CI, and re-pinning is a deliberate, reviewable act.
- **Authorship binding.** The PR author must be a listed author, by GitHub handle, of the

codes they add (`check_authorship.py`, which fails closed on any git error); a contributor who adds a first layout or a first circuit tier to someone else’s entry is bound through that artifact’s own `contributed_by` field, so the code’s author list never changes under them.

- **Duplicate fingerprints.** The reduced-row-echelon forms of (H_X, H_Z) — or of S — pin the exact stabilizer group; an identical fingerprint is a hard CI error, and so is a CSS code that is a known entry up to local Hadamards. A permutation-invariant Weisfeiler–Leman signature on the Tanner graph flags possible equivalents, and for the cyclic two-block family the flag is decided outright by a finite search over ring automorphisms and block shifts rather than left as a review item.
- **An evidence trail a reader can open.** The prose gate (`check_prose.py`) rejects a PR body or research note that cites a path absent from the PR’s own tree, gitignored staging output, a private checkout, leftover drafting scaffolding, or a session URL; an external source must be named and pinned to a commit. A fieldnote added by a PR must credit the PR author’s handle.
- **Local gate for agents.** An autoresearch agent may explore freely, but by convention (`research/AUTORESEARCH.md`) no code is a find until the pinned `validate_candidate` returns `passed: true` — the same verdict CI will reach, computable before a PR is ever opened — and every found witness is persisted through the kit rather than printed and lost.

9 Repository layout and automation

Path	Contents
<code>schema/</code>	submission format (JSON Schema + <code>SCHEMA.md</code>); reproduction and campaign schemas
<code>verify/</code>	the trusted stack (<code>gf2.py</code> , <code>qldpc_verify.py</code> , <code>heuristic_distance.py</code> , <code>circuit_verify.py</code> , <code>ler_verify.py</code> , <code>validate_candidate.py</code> , all hash-pinned); the CI gates (<code>gate_changed.py</code> , <code>scope</code> , <code>authorship</code> , <code>prose</code> , <code>integrity</code>); the offline tiers (<code>certify.py</code> , <code>sat_certify.py</code> , <code>heuristic_certify.py</code>); optional C++ and CUDA search accelerators; tests
<code>codes/</code>	accepted submissions and seeded baselines (the board data)
<code>circuits/</code>	committed memory circuits and detector error models, one directory per entry
<code>certs/</code>	exact and heuristic distance certificates
<code>repro/</code>	reproduction manifests consumed by <code>./qldpc reproduce</code>
<code>decode/</code>	the BP+OSD syndrome-decoder search (offline; no longer part of the gate)
<code>notes/</code>	per-code construction notes (<code>fieldnotes/</code> : dated research write-ups)
<code>research/</code>	contributor tooling: family samplers and screening kit, the open-boundary planar engine, the derived provenance table, and the agent manual
<code>site/</code>	<code>build.py</code> , which generates the static site
<code>docs/</code>	the generated site (index, stabilizer board, per-code pages, references, badges)
<code>cli/</code>	<code>./qldpc submit targets recent reproduce</code>
<code>TRACKS.md</code>	the three ranking layers and how cells work

`CI(.github/workflows/verify.yml)` runs, in order: the scope check, the validator-integrity check, the convention-discovered self-tests (skipped for PRs that only add codes and notes, since those cannot change the tested code), `verify_all.py` over every board code (re-measuring logical error rates only for entries the PR changed, or for every entry when the error-rate stack itself changed),

the authorship check, the refutation gate on the changed codes, and the upload of verification receipts. A separate workflow checks the PR body and changed notes (`prose.yml`), a weekly workflow re-attacks the whole board with fresh seeds (`refute-weekly.yml`), and the site workflow rebuilds `docs/` on every push to `main`. The badges and figures are regenerated from the live board on every build, so they track the current numbers rather than a hand-typed count.

State of the board (snapshot of 1 October 2026). The board holds 1791 verified codes: 1658 CSS codes and 133 general stabilizer codes, of which 120 are seeded literature baselines and the rest were submitted through the challenge. 625 are certified exact; 530 ship a verifier-accepted layout; 48 carry a circuit tier and 26 of those a measured logical error rate. Matched by isomorphism against a literature index maintained alongside the board (the derived table is committed; the index is not), 449 entries reproduce a published code, 255 match published parameters only, and 858 have no match — which means the index holds none, not that the code is unpublished. The previous revision of this document, two months earlier, reported 151 codes and 29 certificates.

The best operational efficiency on the CSS board is $kd^2/n = 1534.4$, held by a $[[682, 172, 78]]$ code (weight-28 checks, so it competes only in the any-weight cell; distance at the upper-bound tier), attributed to the producing LLM in its provenance; the weight-12 record is $[[674, 44, 95]]$ at 589.2. The weight-capped cells tell a more constrained story — $[[904, 230, 22]]$ leads weight-8 at $kd^2/n = 123.1$ and $[[672, 20, 32]]$ leads weight-6 at 30.5 — and the 2D-local cells a more constrained one still: $[[360, 12, 24]]$, a bivariate-bicycle code with a verified bilayer layout, leads both the weight-6 and the weight-8 bilayer cells at 19.2, the exact-certified balanced-product code $[[150, 30, 10]]$ (the instance the mitten and ZSZ-LP papers share) leads any-weight bilayer at 20, and in a single layer $[[288, 8, 15]]$ leads weight-8 at 6.25, $[[454, 8, 17]]$ weight-6 at 5.09, and the exact $[[20, 8, 4]]$ any weight at 6.4. On the geometric side the dense surface-code packings of Section 3.1 now hold every g headline, from 1.68 at $d = 3$ to 1.27 at $d = 13$. The stabilizer board, one week old at this snapshot, is led by $[[66, 12, 14]]$ at $kd^2/n = 35.6$, with $[[144, 10, 16]]$ at 17.8 in weight-8 and $[[85, 8, 9]]$ at 7.6 in weight-6; no stabilizer entry has yet earned a 2D-local class. Every stabilizer distance is an upper bound.

The board’s history is also a record of refutation. Entries that passed the gate at submission and were later tightened by deeper search include $[[882, 18, 30]] \rightarrow 29$ and $[[684, 12, 81]] \rightarrow 66$, and in one agent-driven sweep a two-million-trial rerun refuted five of seven staged $n \geq 300$ frontier candidates *before* submission. The live figures are in `docs/stats.json`.

10 Open questions this could surface

- Can planar (open-boundary) codes close the efficiency gap to the periodic and unrestricted records, or is part of the gap intrinsic to boundaries? The best single-layer cells sit at $kd^2/n \approx 6$ against bilayer cells near 20.
- Dense packings of surface-code patches exceed the surface code’s geometric efficiency at every distance tried, but the margin shrinks with d (1.35 at $d = 5$, 1.27 at $d = 13$). Does the family’s g converge above 1, and does the gain survive at the circuit level once the shared ancillas share hook errors? Is there any *other* honestly laid-out family above 1, with the r^4 price on interaction radius being what every long-range construction must pay down?
- How far does the non-abelian two-block design space reach? Groups outside the systematically enumerated ranges — the board’s $\text{PSL}(2, 7)$ and S_5 entries were first examples, and the mitten and ZSZ-LP families reach rate 1/5 at weight 9 — keep yielding record parameters.
- How deep must a refutation search run before an upper-bound claim at a given n deserves

confidence? The board’s history, including refutations of its own merged entries and staged candidates, is a growing dataset on exactly this question, and the d -only-advance flag now records where the next refutation is most likely.

- How does kd^2/n track measured circuit-level performance? With 26 entries carrying a re-measured logical error rate under one noise model and one decoder, the two frontiers can be compared for the first time.
- Can exact certification follow the board upward? Certificates now reach $n = 726$ and $k = 342$, but none exceeds $d = 14$: the weight cap $d - 1$ is what makes each solve hard, so every record at $d \geq 20$ remains an upper bound. No certifier yet minimizes Pauli weight for general stabilizer codes, and an exact circuit-level tier over H_{dem} is still deferred.

References

- [1] Zijian Liang, Jens Niklas Eberhardt, and Yu-An Chen. Planar quantum low-density parity-check codes with open boundaries. 2025.
- [2] Sergey Bravyi, Andrew W. Cross, Jay M. Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J. Yoder. High-threshold and low-overhead fault-tolerant quantum memory. *Nature*, 627:778–782, 2024.
- [3] Pavel Panteleev and Gleb Kalachev. Degenerate quantum LDPC codes with good finite length performance. *Quantum*, 5:585, 2021.
- [4] Renren Wang and Leonid P. Pryadko. Distance bounds for generalized bicycle codes. *Symmetry*, 14(7):1348, 2022.
- [5] Arda Aydin, Itzhak Tamo, and Alexander Barg. Breaking the bicycle frame: Coset-based quantum LDPC codes, 2026.
- [6] Aditya Bhardwaj, Muzhou Ma, Nadine Meister, Robbie King, Dolev Bluvstein, John Preskill, Madelyn Cain, Qian Xu, and Hsin-Yuan Huang. High-rate qLDPC processors, 2026. arXiv:2607.28795; mitten codes, non-abelian lifted-product qLDPC processors.
- [7] A. Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, 2003.
- [8] Andrew M. Steane. Error correcting codes in quantum theory. *Physical Review Letters*, 77(5):793–797, 1996.
- [9] Hsiang-Ku Lin and Leonid P. Pryadko. Quantum two-block group algebra codes. *Physical Review A*, 109(2):022407, 2024.
- [10] Sergey Bravyi, David Poulin, and Barbara Terhal. Tradeoffs for reliable quantum information storage in 2D systems. *Physical Review Letters*, 104(5):050503, 2010. arXiv:0909.5200.
- [11] Pavel Panteleev and Gleb Kalachev. Asymptotically good quantum and locally testable classical ldpc codes. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 375–388, 2022.
- [12] Jean-Pierre Tillich and Gilles Zémor. Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength. *IEEE Transactions on Information Theory*, 60(2):1193–1202, 2014.
- [13] Vincent Steffan, Shin Ho Choe, Nikolas P. Breuckmann, Francisco Revson Fernandes Pereira, and Jens Niklas Eberhardt. Tile codes: High-efficiency quantum codes on a lattice with boundary, 2025.
- [14] Yifan Hong. Quantum LDPC codes with design rate 1/5 and good performance below 1000 physical qubits, 2026. arXiv:2607.27644; ZSZ-LP balanced-product codes over metacyclic groups.
- [15] Michael A. Perlin. qLDPC. <https://github.com/qLDPCOrg/qLDPC>, 2023.

- [16] Upendra Kapshikar and Srijita Kundu. On the hardness of the minimum distance problem of quantum codes. *IEEE Transactions on Information Theory*, 69(10), 2023. arXiv:2203.04262.
- [17] Leonid P. Pryadko, Vadim A. Shabashov, and Valerii K. Kozin. QDistRnd: A GAP package for computing the distance of quantum error-correcting codes. *Journal of Open Source Software*, 7(71):4120, 2022.
- [18] Joschka Roffe, David R. White, Simon Burton, and Earl Campbell. Decoding across the quantum low-density parity-check code landscape. *Physical Review Research*, 2(4):043423, 2020.
- [19] Juan Cruz-Benito, Andrew W. Cross, David Kremer, and Ismael Faro. Evolutionary discovery of bivariate bicycle codes with llm-guided search, 2026.
- [20] Austin Yubo He and Zi-Wen Liu. Discovering highly efficient low-weight quantum error-correcting codes with reinforcement learning, 2025. arXiv:2502.14372.
- [21] Zidu Liu and Florian Marquardt. Large-language-model discovery of quantum LDPC codes through structured concept evolution, 2026. arXiv:2606.24808.